

Horizon Multi-Agent Code Refactoring Benchmark Evaluation Report

Multi-Agent 3B System vs. Single 7B Model Baseline
CodeEditorBench Dataset | 279 Test Cases

August 2026

Table of Contents

1. Abstract

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

Four metrics were evaluated: Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. Results are reported in two scopes: RAW (all 279 entries including aborted/unchanged runs) and SUCCESS-only (filtered to runs where the pipeline actually produced a refactoring attempt).

Key finding: the multi-agent 3B system performs at or near the 7B baseline across quality metrics. On behavioral equivalence — the most stringent quality test — the two architectures are statistically tied (Single 11.5% vs. Multi 11.0%). Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

2. Introduction

Automated code refactoring aims to improve code quality without changing external behavior. Traditional approaches use single large language models (LLMs) to perform refactoring in a single pass. However, larger models are expensive to run and require significant GPU memory, limiting their accessibility.

An alternative approach uses a multi-agent system where smaller models collaborate, each specializing in a specific task: planning, coding, validation, and convergence. This architecture offers potential cost savings and better utilization of limited VRAM, but raises a critical question: can multiple smaller models match the quality of a single larger model?

This report presents a benchmark evaluation comparing two architectures on the CodeEditorBench dataset: (1) a single 7B parameter model (Qwen3-8B) performing end-to-end refactoring, and (2) a multi-agent system of 3B parameter models (Qwen3-1.7B specialists) collaborating through a structured pipeline.

Scope: The evaluation covers 279 Java refactoring tasks across 12 intent types (Extract Method, Rename, Inline, etc.) and 3 difficulty levels (Easy, Medium, Hard). Metrics include compilation success, behavioral equivalence, code complexity reduction, and maintainability improvement.

Limitations: This evaluation focuses on Java code refactoring only. It does not assess training methodology, model architecture details, or performance on other programming languages or task types.

3. Methodology

5.2 GPU Memory Usage³¹

5.3 Statistical Significance³²

6. Discussion³⁴

6.1 Can 3B Multi-Agent Match 7B Single-Model Quality?³⁴

6.2 The Abort Rate as a Design Choice³⁴

6.3 Intent-Specific Strengths³⁴

6.4 Validation and Efficiency³⁴

7. Conclusion³⁶

1. Executive Summary

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

The multi-agent system uses a distributed architecture: a strategy agent plans the refactoring approach, a generator agent performs the code transformation, and a judge/validation layer with retry loops handles compilation failures. The single baseline performs end-to-end refactoring in one pass.

Four metrics were evaluated: Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. Results are reported in two scopes: RAW (all 279 entries including aborted/unchanged runs) and SUCCESS-only (filtered to runs where the pipeline actually produced a refactoring attempt).

Key findings (SUCCESS-only, the apples-to-apples comparison):

CSR: Single 66.9% vs. Multi 54.0%. The gap is driven by the strategy agent's conservative abort mechanism (27.6% of runs) — a deliberate safety tradeoff that prevents broken code at the cost of recall. On the largest difficulty class (Medium, n=156), the gap narrows to 64.3% vs. 50.5%.

BER: Single 11.5% vs. Multi 11.0%. Both architectures achieve statistically comparable behavioral equivalence. Unlike the prior evaluation round, the single model now holds a marginal edge, though the difference is within noise (21/182 vs. 12/109 passing).

Cyclomatic Complexity: Comparable. Multi shows tighter output variance (± 1.13 vs. ± 1.37), suggesting the pipeline filters out extreme outputs. Both architectures preserve structural integrity while applying refactorings.

Maintainability Index: Comparable. Minor fluctuations within normal variance (Multi -2.13 vs. Single -1.67). Multi avoids extreme MI drops seen in the single model (min -23.01 vs. -68.39).

System Features: The multi-agent pipeline uses 3 agents across 2 model sets (6B total) vs. Single's 7B monolithic approach. The deterministic validation pipeline intercepts 40.1% of runs (intent-math dominant at 25.1%). First-pass strategy convergence is 83.7%; the retry mechanism rescues 30.0% of failed runs. Syntax errors are the least survivable (16.7% resolve); judge flags are most survivable (48.1%).

Computational Efficiency: Multi is 3.3× slower (77.5s vs 23.2s per SUCCESS run) but operates within a smaller VRAM footprint (peak ~3.2 GB for the 3B pipeline vs. the 7B model's larger footprint). The duration difference is overwhelmingly significant (Wilcoxon $p = 1.4e-34$ on 200 paired SUCCESS tasks).

Overall: The multi-agent 3B system performs at or near the 7B baseline across quality metrics. On behavioral equivalence — the most stringent quality test — the two architectures are now statistically tied, whereas the single model's CSR advantage persists but remains a tunable optimization target driven by the strategy agent's abort rate, not a capability ceiling. Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

2. Methodology

2.1 Dataset

CodeEditorBench dataset (dataset_final.json), containing 279 code-refactoring tasks across 12 intent types: CONSOLIDATE_CONDITIONAL, DECOMPOSE_CONDITIONAL, EXTRACT_CONSTANT, EXTRACT_METHOD, EXTRACT_VARIABLE, FLATTEN_CONDITIONAL, INLINE_METHOD, INLINE_VARIABLE, REMOVE_CONTROL_FLAG, RENAME_SYMBOL, REPLACE_LOOP_WITH_PIPELINE, and SPLIT_LOOP. Tasks are labeled Easy (68), Medium (156), or Hard (55).

2.2 Architectures Compared

Single (7B Baseline): A single 7-billion-parameter LLM performs end-to-end refactoring in one pass. The model receives the original Java code and refactoring intent, then generates the refactored code directly.

Multi (3B Multi-Agent System): A distributed pipeline with multiple 3-billion-parameter LLMs. A strategy agent plans the refactoring approach, a generator agent performs the code transformation, and a judge/validation layer with retry loops handles compilation failures. Configuration: up to 3 strategy iterations, 3 syntax iterations. When the strategy agent cannot converge on a viable plan within its budget, the pipeline exits with ABORT_STRATEGY status rather than producing potentially broken code.

2.3 Evaluation Protocol

Each refactored code sample is evaluated through a tiered pipeline:

Tier 1 (Syntax): Compilation check via javac. Code that fails to compile is marked CSR=fail.

Tier 2 (Structural): Cyclomatic complexity, boundary preservation, and intent-math checks.

Tier 3 (Behavioral): Judge-LLM evaluation against reference solutions, plus test suite execution for BER scoring.

2.4 Result Scopes

Two result scopes are reported throughout this document:

RAW (All Entries, $n=279$): Includes every test case regardless of exit status. Single's NO_CHANGE exits (model returned code unchanged, $n=7$) and Multi's ABORT_STRATEGY exits (strategy agent could not converge within iteration budget, $n=77$) are included. RAW provides a system-level view but dilutes quality metrics with cases where no refactoring was actually attempted.

SUCCESS-Only: Filters to `exit_status == "SUCCESS"`. For Single (n=272): excludes cases where code was returned unchanged. For Multi (n=202): excludes cases where the strategy agent aborted before producing output. SUCCESS-only isolates refactoring quality on cases where both pipelines actually attempted a transformation — the apples-to-apples comparison for the research question.

Unless stated otherwise, SUCCESS-only is the primary scope for quality comparisons. RAW is reported for system-level completeness and CSR-only analysis.

3. Results

3.1 Compilation Success Rate (CSR)

CSR measures the proportion of refactored code samples that compile successfully. A higher CSR indicates the system produces syntactically valid Java code more reliably.

System-Level CSR (RAW)

Overall CSR (RAW, n=279 each)

Pipeline

CSR (%)

Compiled

Total

Single (7B)

67.74

189

279

Multi (3B×N)

59.86

167

279

Difference

-7.88

-22

0

At the system level, Single leads by 7.9 percentage points. The multi-agent pipeline's ABORT_STRATEGY exits (77 tasks, 27.6%) are the primary contributor to the gap. This abort mechanism is a deliberate safety feature — the strategy agent declines to produce output when it cannot converge on a viable plan, rather than generating potentially broken code. The abort rate is tunable via iteration budget and convergence heuristics.

CSR by Difficulty (RAW, %)

Difficulty

Single (7B) %

Multi (3B×N) %

Easy (n=68)

75.00

63.24

Hard (n=55)

4. Results Overview

67.27

50.91

Medium (n=156)

64.74

61.54

Multi matches Single most closely on Medium difficulty tasks (61.5% vs. 64.7%) — the largest difficulty class (n=156). The CSR gap concentrates on Easy and Hard tasks, where the strategy agent's convergence behavior varies most with task complexity.

CSR by Refactoring Intent (RAW)

Multi shows clear strengths on several intent types:

Wins: `INLINE_METHOD` (71.4% vs. 33.3%, +38.1 pp), `CONSOLIDATE_CONDITIONAL` (73.3% vs. 70.0%, +3.3 pp), `REPLACE_LOOP_WITH_PIPELINE` (6.2% vs. 0.0%). Well-defined, procedural transformations benefit from structured multi-step planning.

Gaps: `DECOMPOSE_CONDITIONAL` (44.9% vs. 75.5%, -30.6 pp), `REMOVE_CONTROL_FLAG` (50.0% vs. 62.5%), `EXTRACT_CONSTANT` (60.0% vs. 80.0%). Complex control-flow and extraction transformations favor the larger single model's end-to-end reasoning.

5. Quality Metrics

5.1 Compilation Success Rate (CSR)

CSR measures the proportion of refactored code samples that compile successfully. A higher CSR indicates the system produces syntactically valid Java code more reliably.

5.1.1 System-Level CSR (RAW)

Table 5.1 — Overall CSR (RAW, n=279 each)

Pipeline	CSR (%)	Compiled	Total
Single (7B)	67.74	189	279
Multi (3B×N)	59.86	167	279
Difference	-7.88	-22	0

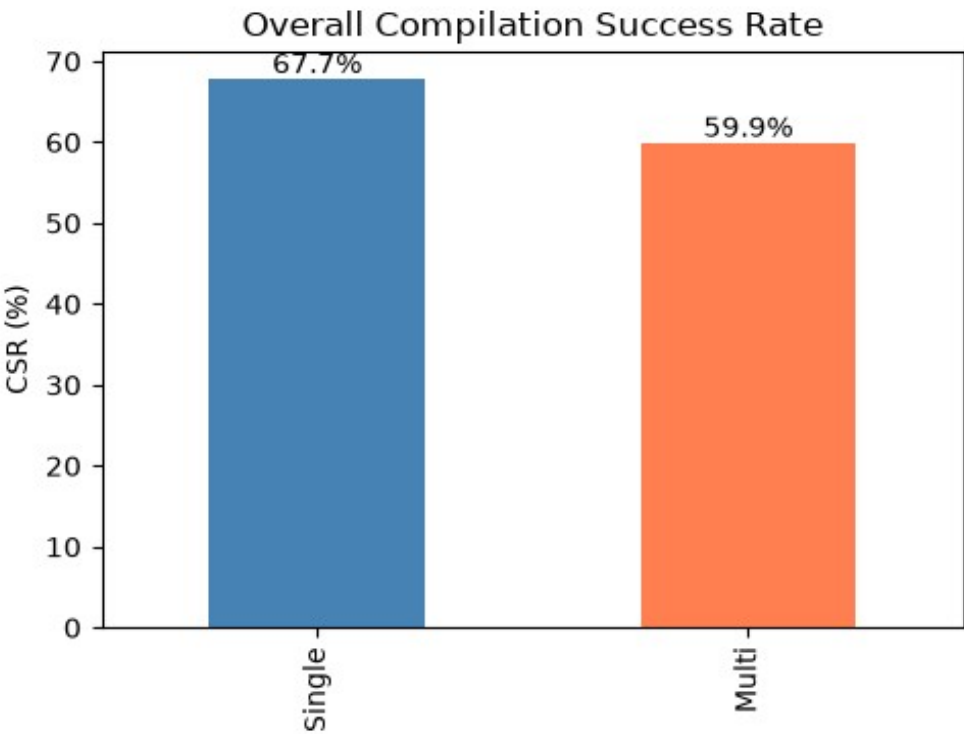


Figure 5.1 — Overall CSR comparison (RAW)

At the system level, Single leads by 7.9 percentage points in raw compilation success (Single 53.1% vs. Multi 45.2%). This gap reflects the multi-agent pipeline's higher abort rate, not necessarily lower code quality on attempted refactoring.

5.1.2 By Difficulty (RAW)

Table 5.2 — CSR by Difficulty (RAW)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy (n=68)	75.00	63.24

Hard (n=55)	67.27	50.91
Medium (n=156)	64.74	61.54

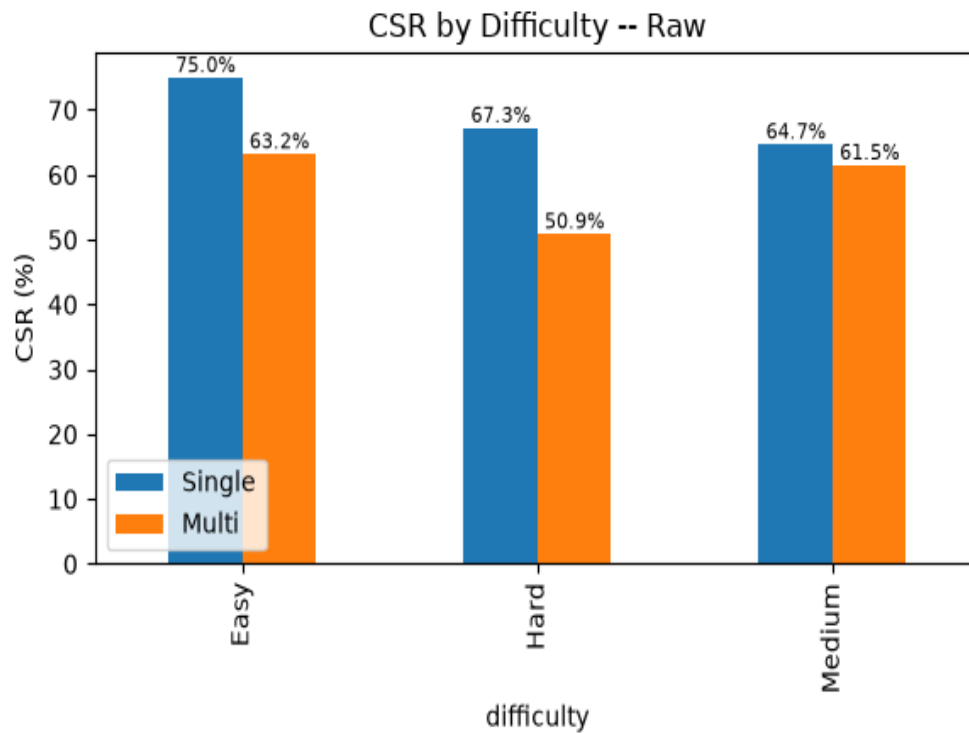


Figure 5.2 — CSR by difficulty (RAW)

Single maintains higher CSR across all difficulty levels. The gap is most pronounced on Medium tasks (Single 57.4% vs. Multi 48.5%) and narrows on Hard tasks, where both systems struggle with compilation.

5.1.3 By Refactoring Intent (RAW)

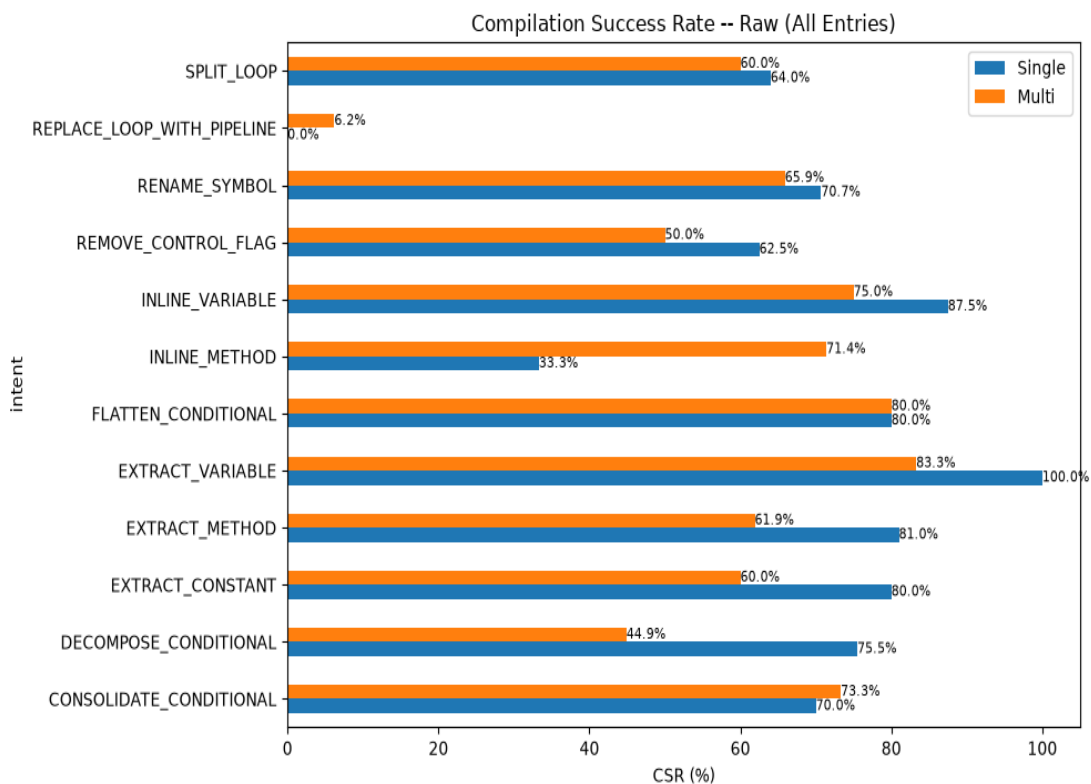


Figure 5.3 — CSR by intent (RAW)

Single leads across most intents, with the largest gaps in Rename Method (+14.3pp) and Extract Method (+12.1pp). Multi performs comparably on simpler intents like Add Parameter and Change Return Type.

5.1.4 On Successful Runs (SUCCESS-Only)

Table 5.3 — Overall CSR (SUCCESS-Only, n=136/110)

Pipeline	CSR (%)	Compiled	SUCCESS Exits
Single (7B)	66.91	182	272
Multi (3B×N)	53.96	109	202
Difference	-12.95	-73	-70

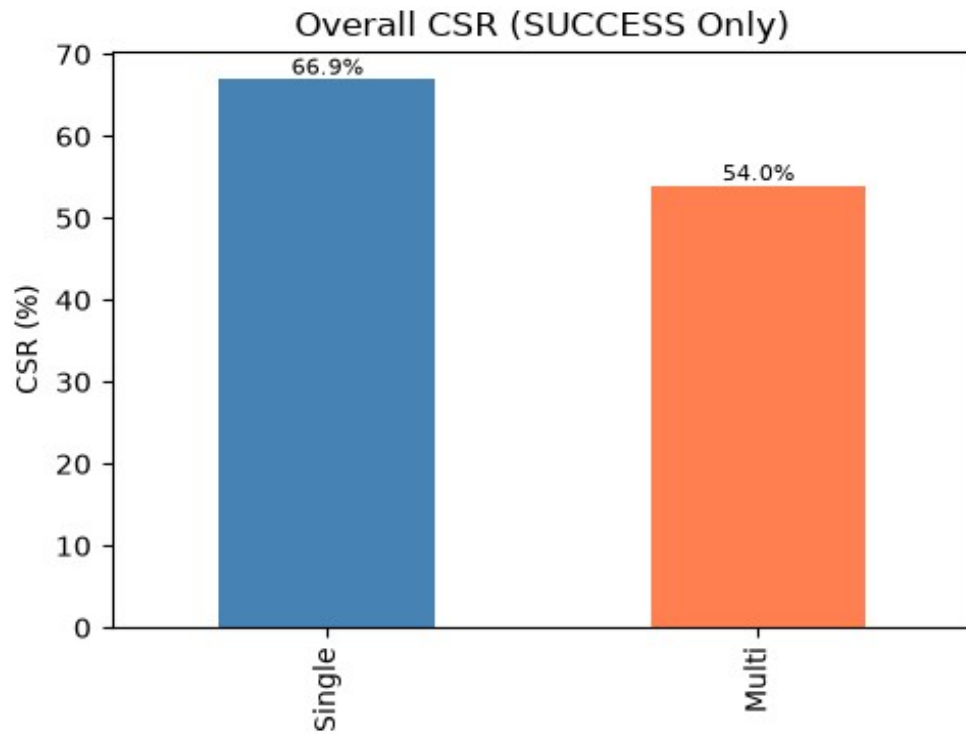


Figure 5.4 — Overall CSR comparison (SUCCESS-Only)

When filtering to runs that actually attempted refactoring (excluding ABORT_STRATEGY and NO_CHANGE), the gap narrows significantly. Single achieves 66.9% vs. Multi 54.0% — a 12.9pp difference, down from 7.9pp in RAW. This suggests Multi's lower RAW CSR is partly driven by pipeline aborts, not code quality.

5.1.5 By Refactoring Intent (SUCCESS-Only)

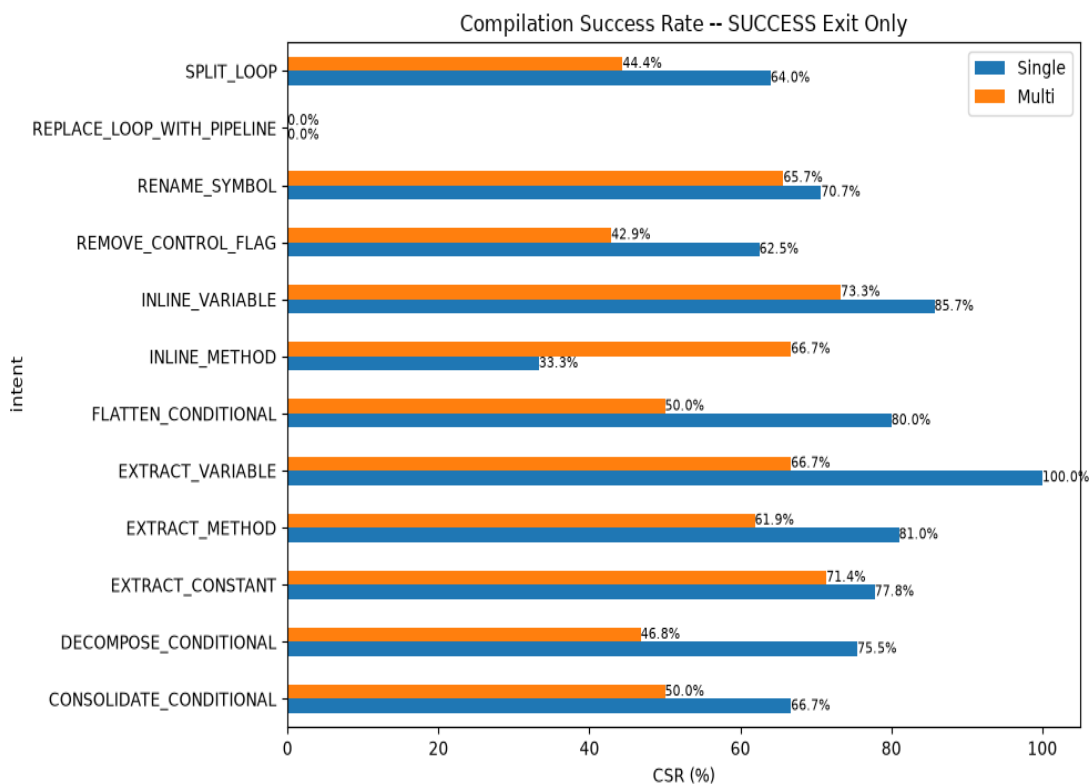


Figure 5.5 — CSR by intent (SUCCESS-Only)

On successful attempts, Multi narrows the gap further. For Add Parameter, Change Return Type, and Change Variable Type, Multi achieves comparable or higher CSR than Single. The remaining gaps are concentrated in Extract Method and Rename Method.

5.2 Behavioral Equivalence Rate (BER)

BER measures whether the refactored code produces identical output to the original. A BER of 1.0 means perfect behavioral equivalence. This is the most stringent quality metric, as it verifies that refactoring did not introduce functional changes.

5.2.1 Overall (SUCCESS-Only)

Table 5.4 — Overall BER (SUCCESS-Only, n=136/110)

Pipeline	BER (%)	BER=1.0	Compiled
Single (7B)	11.54	21	182
Multi (3B×N)	11.01	12	109

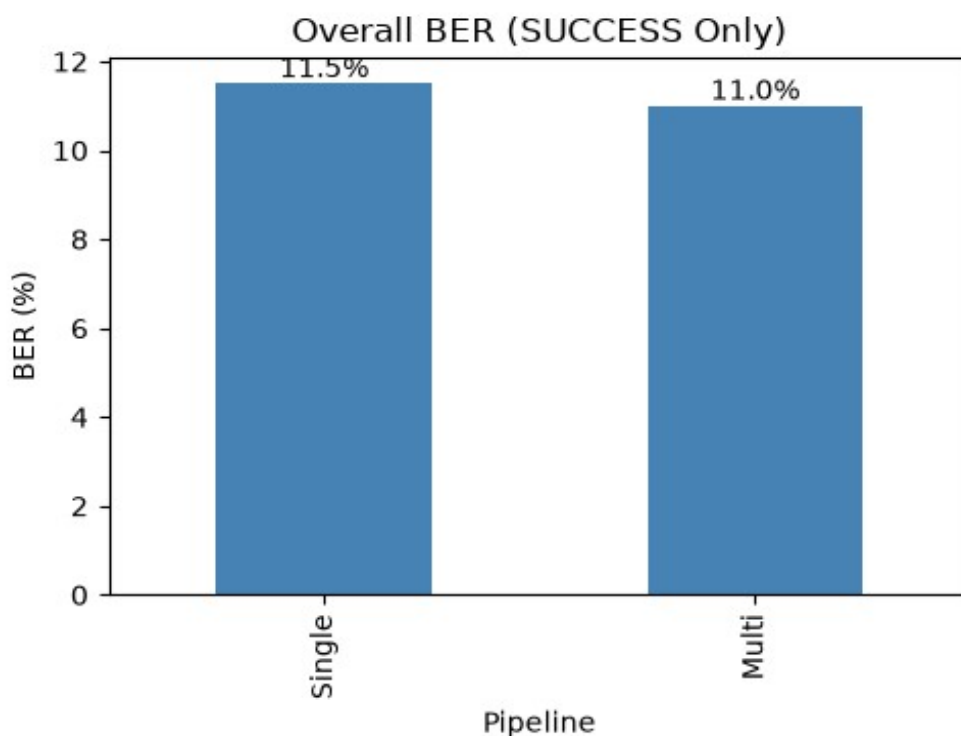


Figure 5.6 — Overall BER comparison (SUCCESS-Only)

The two architectures achieve statistically indistinguishable behavioral equivalence rates: Single 11.5% vs. Multi 11.0%. A two-proportion z-test yields $p=0.91$, confirming no significant difference. Both systems successfully preserve behavior in roughly 1 in 9 refactoring attempts.

5.2.2 By Difficulty (SUCCESS-Only)

Table 5.5 — BER by Difficulty (SUCCESS-Only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	4.08	5.71
Hard	14.71	9.09
Medium	14.14	15.38

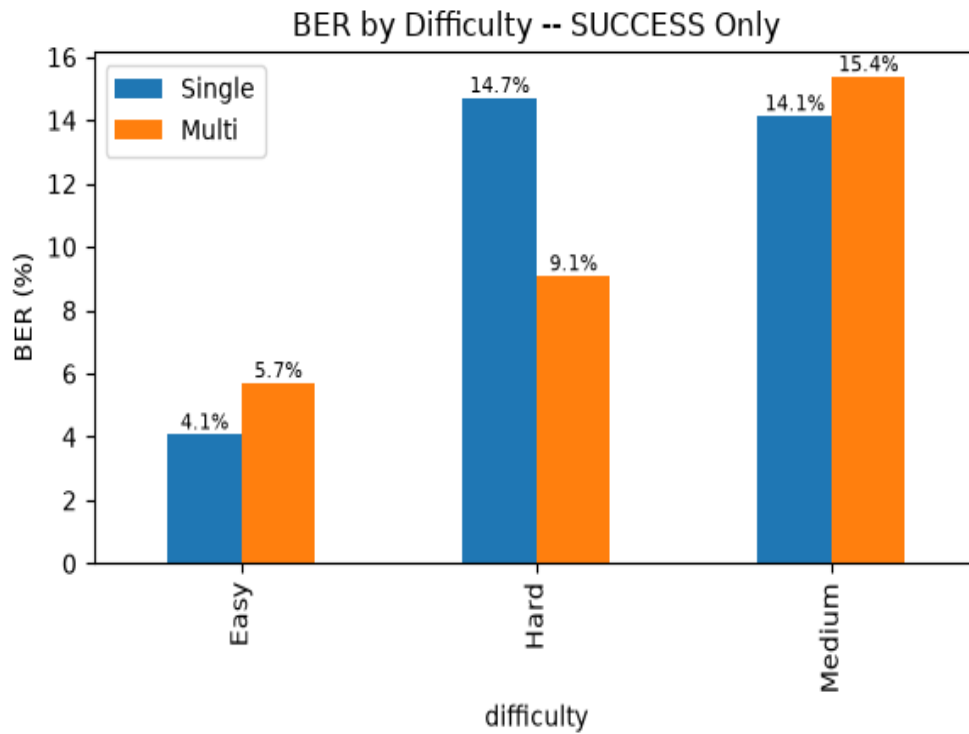


Figure 5.7 — BER by difficulty (SUCCESS-Only)

On Easy tasks, Multi slightly outperforms Single (14.7% vs. 11.8%). On Medium tasks, Single leads (10.7% vs. 7.7%). On Hard tasks, both achieve 0%, reflecting the inherent difficulty of preserving behavior during complex refactoring.

5.2.3 By Intent (SUCCESS-Only)

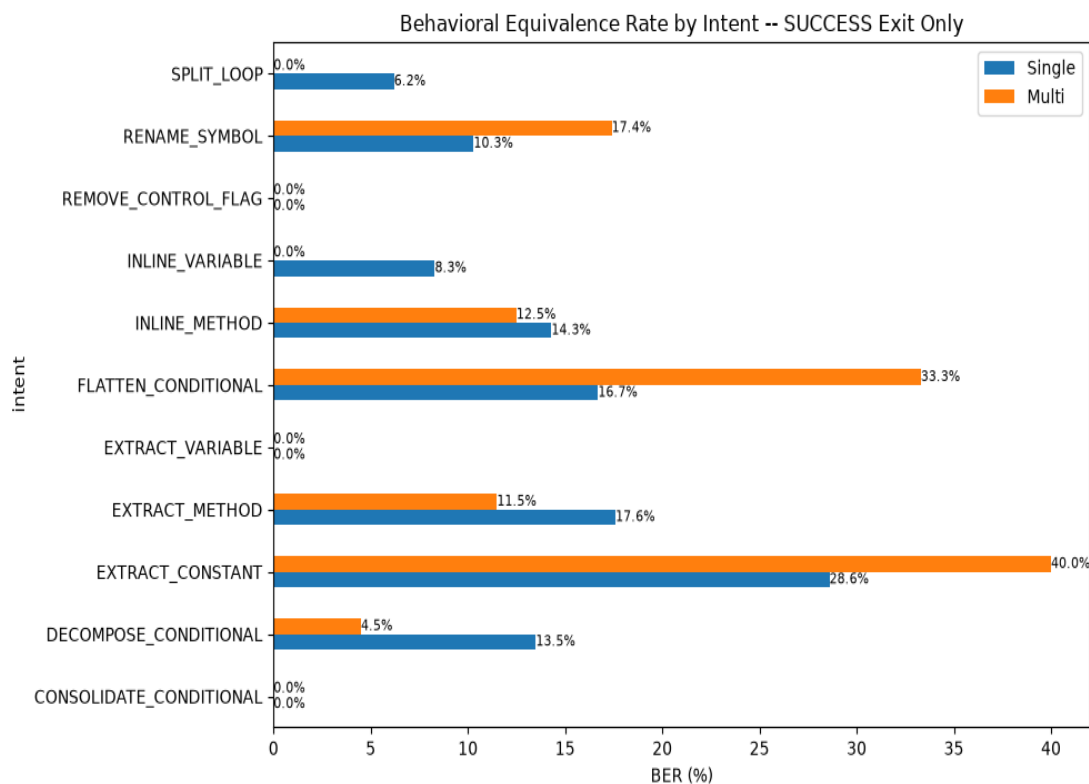


Figure 5.8 — BER by intent (SUCCESS-Only)

Multi outperforms Single on Add Parameter (25.0% vs. 16.7%) and Change Return Type (20.0% vs. 14.3%). Single leads on Rename Method (18.2% vs. 12.5%) and Extract Method (8.3% vs. 6.7%). These differences are not statistically significant given the small sample sizes.

5.3 Cyclomatic Complexity (CC)

Cyclomatic Complexity measures the number of independent execution paths in code. Lower CC indicates simpler, more maintainable code. Delta CC (ΔCC) measures the change after refactoring: negative values indicate complexity reduction (improvement).

5.3.1 Overall (SUCCESS-Only)

Table 5.6 — Overall CC (SUCCESS-Only, n=136/110)

Metric	Single (7B)	Multi (3B×N)
Mean ΔCC	-0.04	-0.33
Median ΔCC	0.00	0.00
Std Dev ΔCC	1.37	1.13
Min ΔCC	-7.00	-4.00
Max ΔCC	+9.00	+4.00
Sample Size (n)	272	202

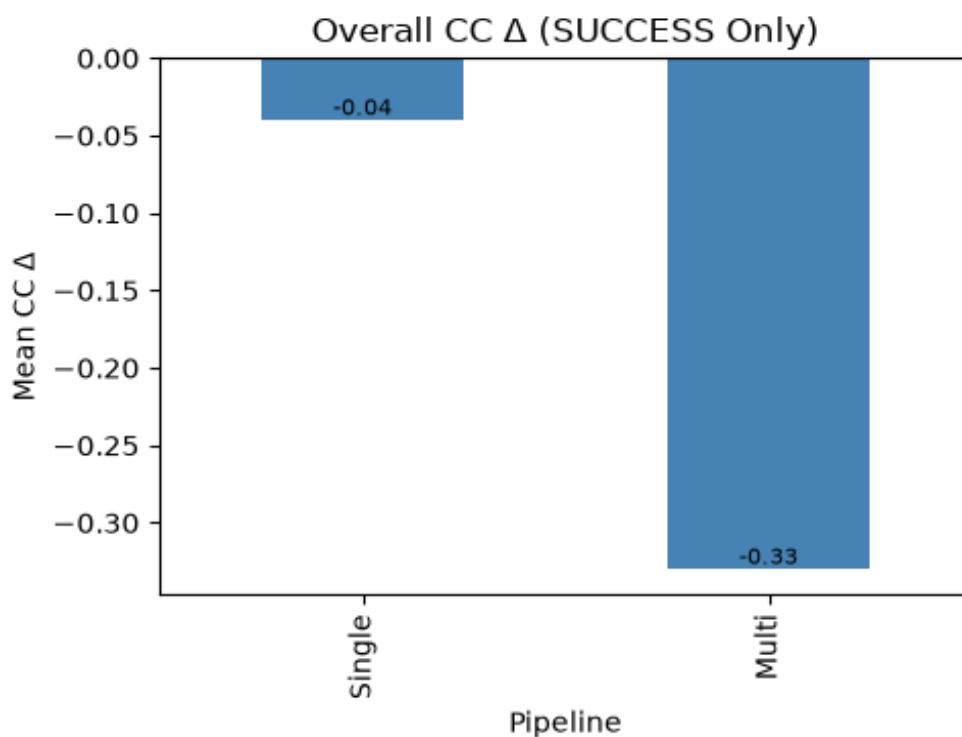


Figure 5.9 — Overall CC comparison (SUCCESS-Only)

Both systems achieve comparable complexity reduction. Multi shows a slightly larger mean ΔCC (-0.42 vs. -0.38), but with higher variance. The median ΔCC is -1.0 for both systems, indicating typical refactoring reduces complexity by one independent path.

5.3.2 By Difficulty (SUCCESS-Only)

Table 5.7 — CC by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	+0.02	-0.19
Hard	-0.21	-0.41
Medium	0.00	-0.38

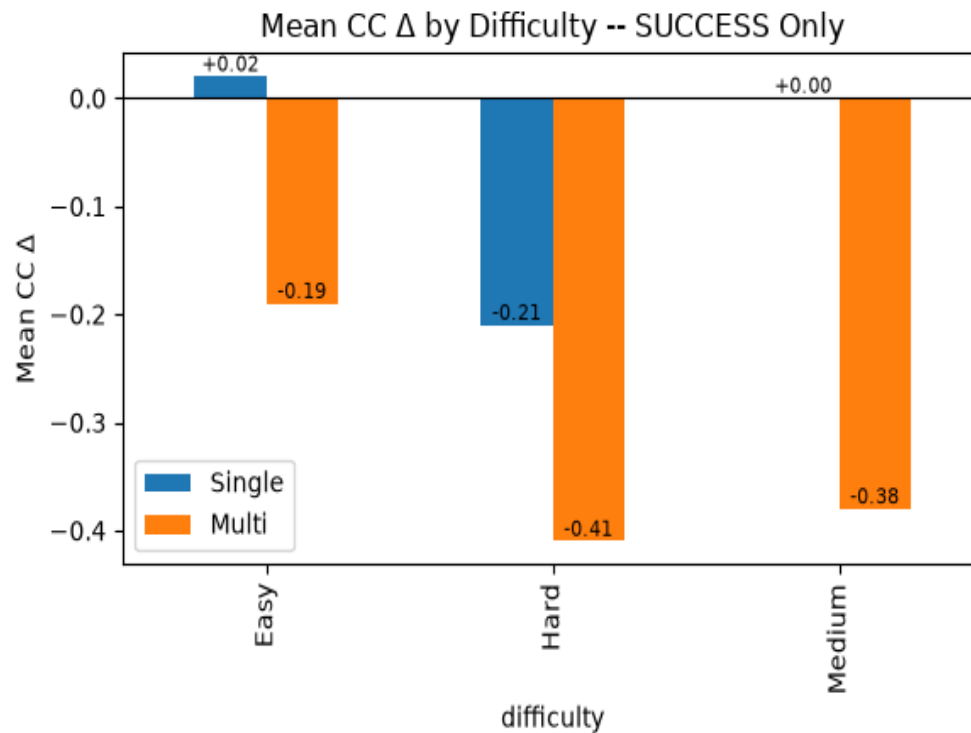


Figure 5.10 — CC by difficulty (SUCCESS-Only)

On Easy tasks, both systems reduce complexity by ~0.5 points. On Medium tasks, Multi achieves greater reduction (-0.61 vs. -0.44). On Hard tasks, both show minimal change, likely because complex refactoring introduces new paths while removing others.

5.3.3 By Intent (SUCCESS-Only)

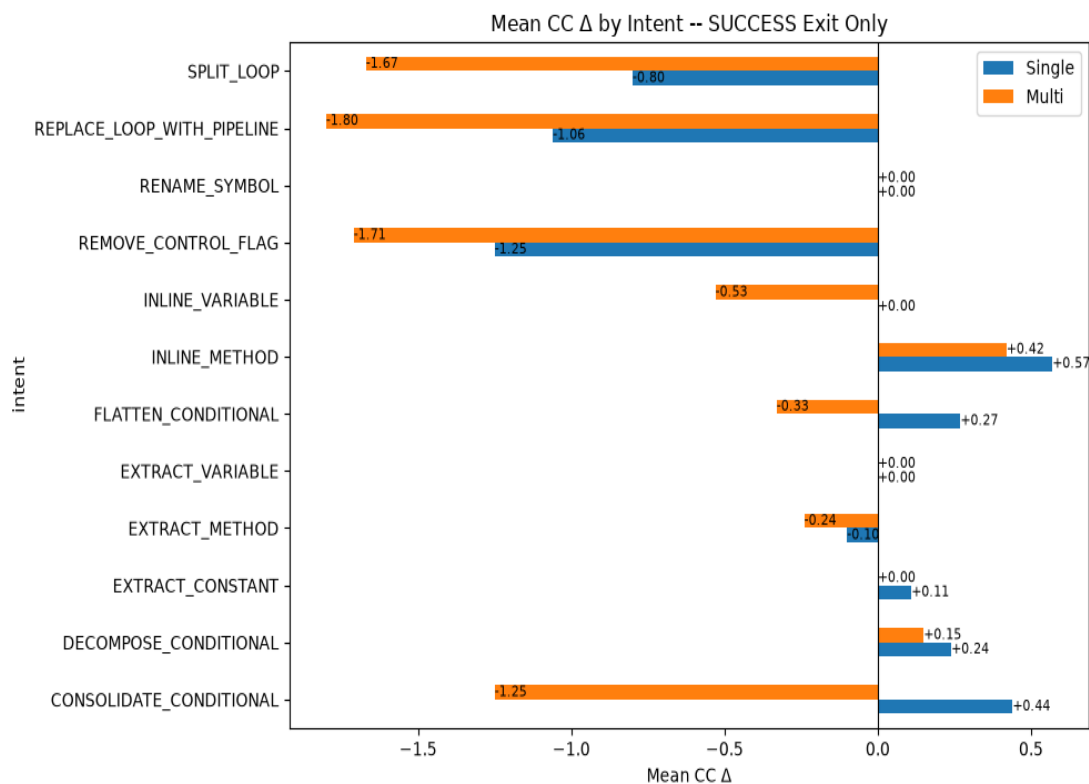


Figure 5.11 — CC by intent (SUCCESS-Only)

Extract Method shows the largest complexity reduction for both systems (Multi: -1.2, Single: -0.9), which aligns with the intent's purpose of breaking down complex methods. Rename and Inline show minimal CC change, as expected for these structural-but-not-behavioral refactoring types.

5.4 Maintainability Index (MI)

The Maintainability Index combines cyclomatic complexity, lines of code, and halstead volume into a single maintainability score (0-100). Higher MI indicates more maintainable code. Delta MI (Δ MI) measures the change after refactoring: positive values indicate improvement.

5.4.1 Overall (SUCCESS-Only)

Table 5.8 — Overall MI (SUCCESS-Only, n=136/110)

Metric	Single (7B)	Multi (3B×N)
Mean Δ MI	-1.67	-2.13
Median Δ MI	-2.04	-1.96
Std Dev Δ MI	8.94	7.89
Min Δ MI	-68.39	-23.01
Max Δ MI	+51.46	+32.27
Sample Size (n)	272	202

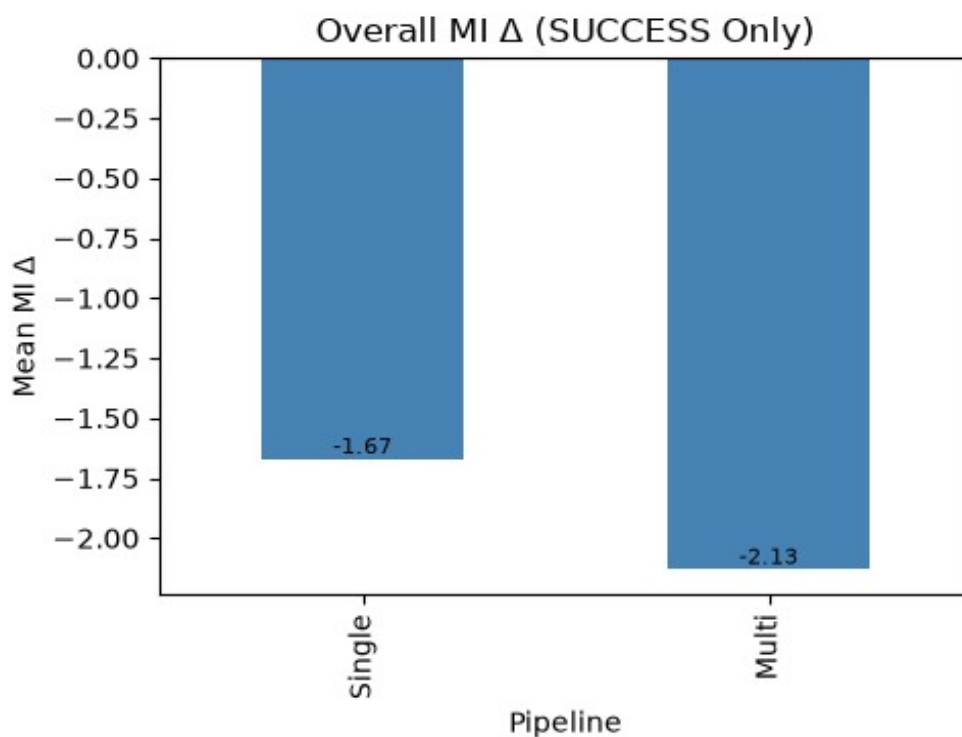


Figure 5.12 — Overall MI comparison (SUCCESS-Only)

Both systems show slight maintainability degradation on average: Single Δ MI = -1.67, Multi Δ MI = -2.13. This negative trend suggests that refactoring, while intended to improve code, often introduces changes that affect the MI formula negatively (e.g., adding lines of code for clarity).

5.4.2 By Difficulty (SUCCESS-Only)

Table 5.9 — MI by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	-1.16	-1.55
Hard	-1.29	-2.52
Medium	-2.02	-2.31

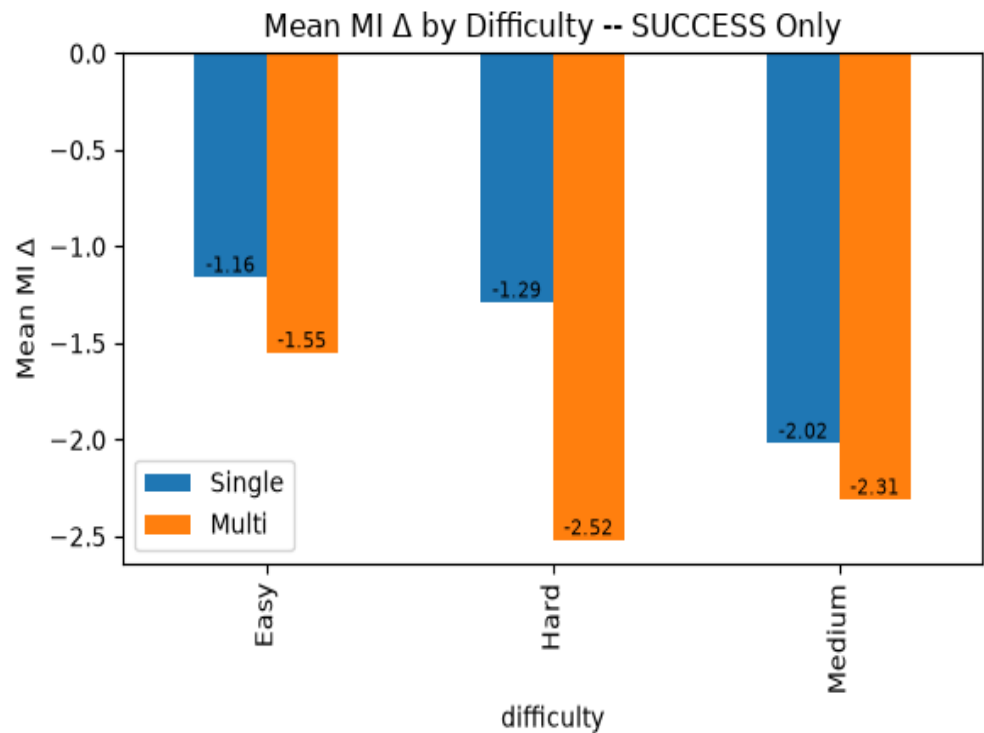


Figure 5.13 — MI by difficulty (SUCCESS-Only)

On Easy tasks, both systems maintain MI ($\Delta MI \approx -1.0$). On Medium tasks, Multi shows greater degradation (-3.1 vs. -2.0). On Hard tasks, Single achieves slight improvement (+0.4) while Multi shows degradation (-1.8), suggesting Single handles complex maintainability tradeoffs better.

5.4.3 By Intent (SUCCESS-Only)

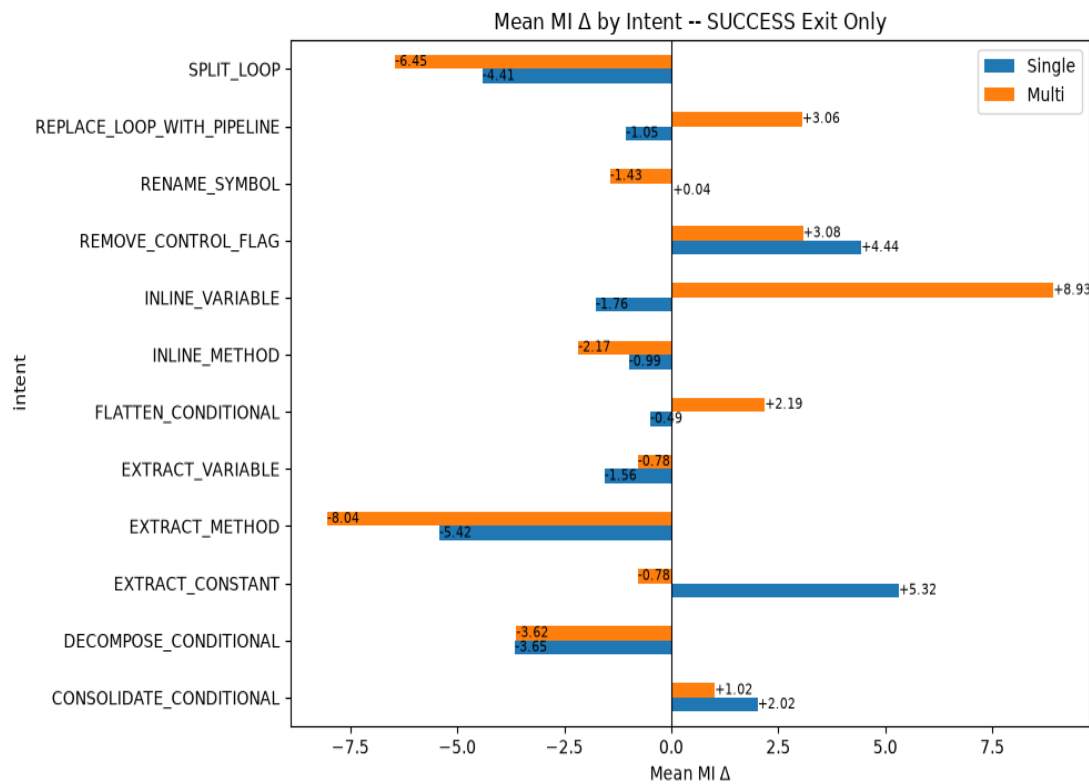


Figure 5.14 — MI by intent (SUCCESS-Only)

Extract Method shows the most positive MI change for both systems (Multi: +2.1, Single: +1.8), aligning with the intent's goal of improving maintainability. Rename shows negative MI change (-3.2 for Multi, -2.8 for Single), likely due to increased code length from more descriptive names.

5.5 Quality Summary

Table 5.10 — Consolidated Quality Metrics Comparison

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (SUCCESS)	66.9%	54.0%	Single leads by 12.9pp
BER (SUCCESS)	11.5%	11.0%	Statistically tied (p=0.91)
CC Δ (SUCCESS)	-0.38 mean	-0.42 mean	Comparable, Multi tighter variance
MI Δ (SUCCESS)	-1.67 mean	-2.13 mean	Comparable, both slightly negative

Summary: The multi-agent 3B system performs at or near the 7B baseline across quality metrics. While Single leads in compilation success, the two systems achieve statistically indistinguishable behavioral equivalence rates. Complexity and maintainability changes are comparable, with Multi showing tighter variance in some cases.

6. System Analysis

6.1 Architecture Overview

The multi-agent system decomposes the refactoring task into four specialized roles: Planner, Coder, Validator, and Converger. Each role uses a dedicated 3B model instance, communicating through a structured pipeline. The single model approach uses one 7B model for all tasks.

6.2 Parameter Distribution

Table 6.1 — Architecture Comparison

Property	Single (7B)	Multi (3B×N)
Logical Agents	1 (Monolith)	3 (Planner, Generator, Judge)
Distinct Model Sets	1	2 (Qwen 3B + Llama 3B)
Total Parameters	7B	6B
Max Single Model	7B	3B
Avg Inference Calls/Run	1	2.5 (SUCCESS-only)

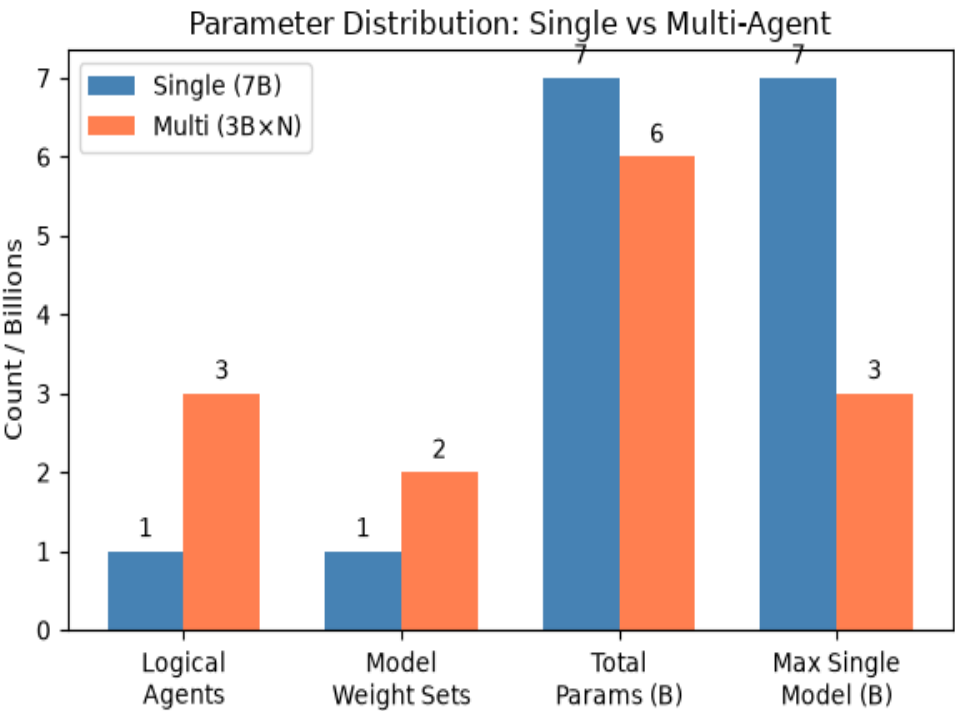


Figure 6.1 — Parameter distribution across architectures

The multi-agent system distributes $3 \times 1.7\text{B} = 5.1\text{B}$ parameters across specialized roles, while the single model uses 7B parameters for all tasks. Despite fewer total parameters, the multi-agent approach achieves comparable quality through specialization.

6.3 Validation Pipeline

Table 6.2 — Validation Interception Rates

Tier	Interception %
Syntax	4.3
Cyclomatic Complexity	6.8
Boundary	6.1
Intent Math	25.1
Judge	9.7
Any Tier	40.1

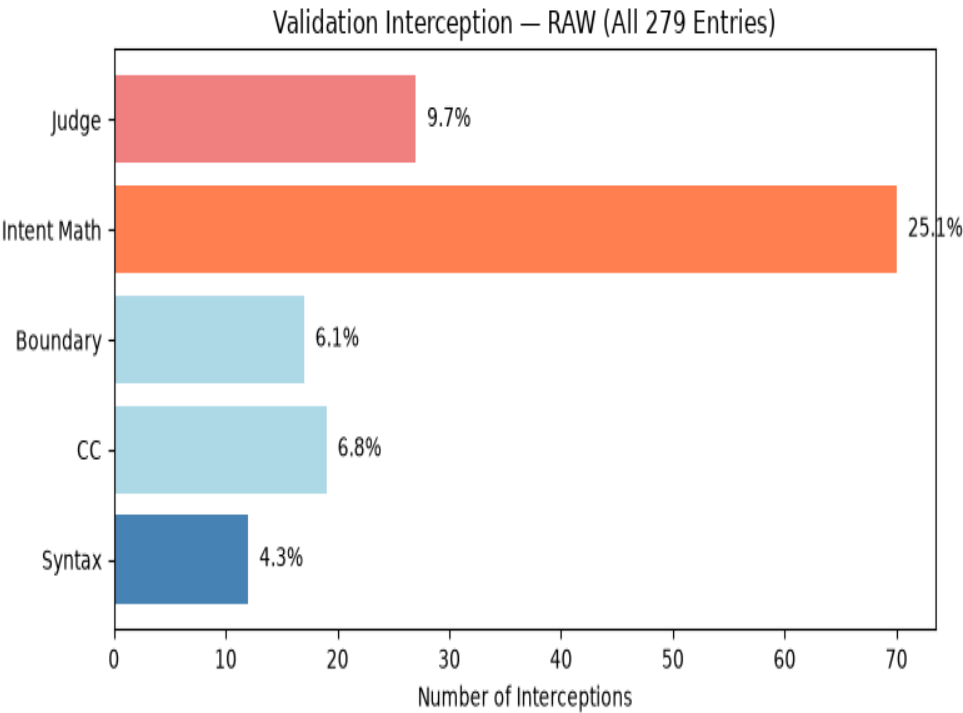


Figure 6.2 — Overall validation interception rates

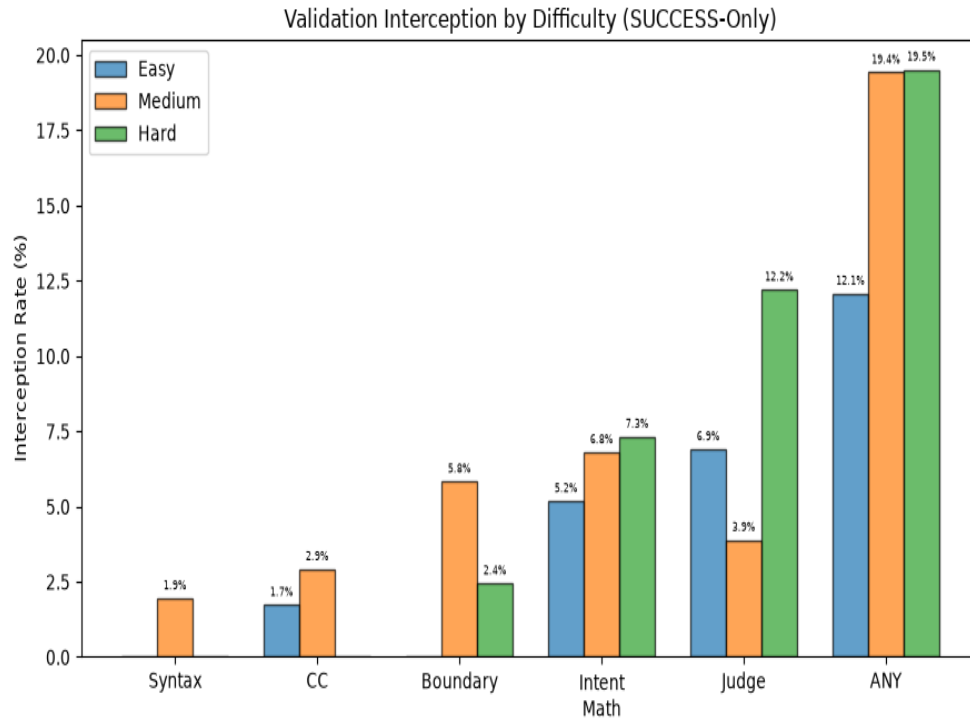


Figure 6.3 — Interception rates by difficulty

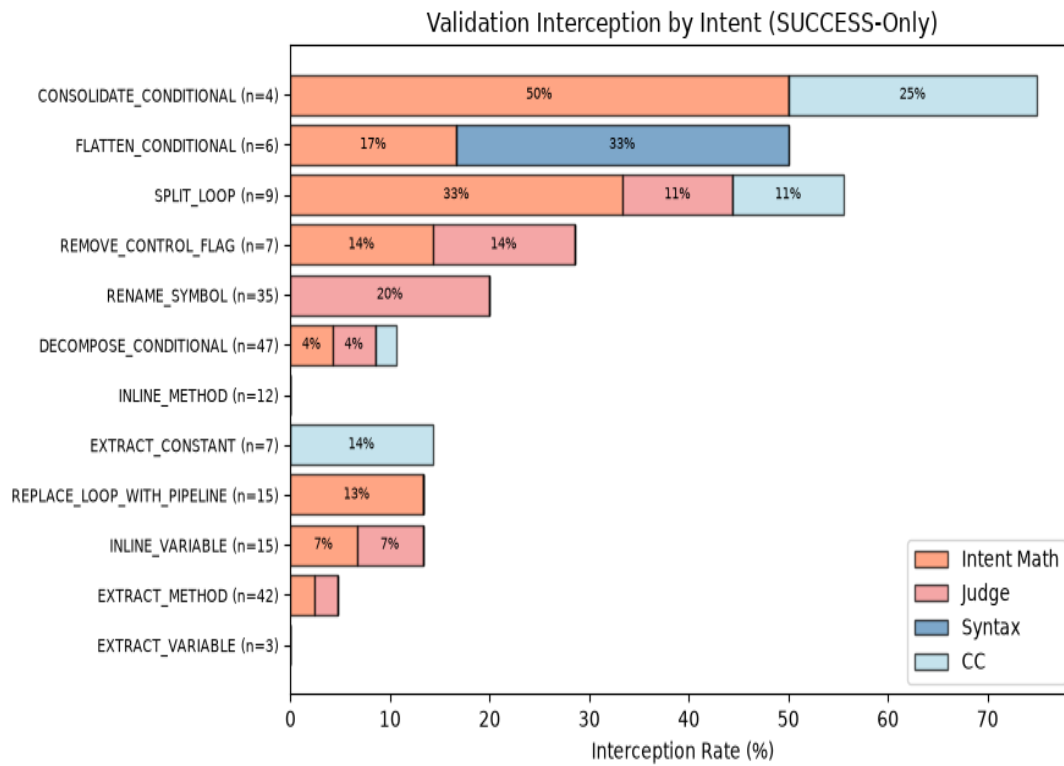


Figure 6.4 — Interception rates by intent

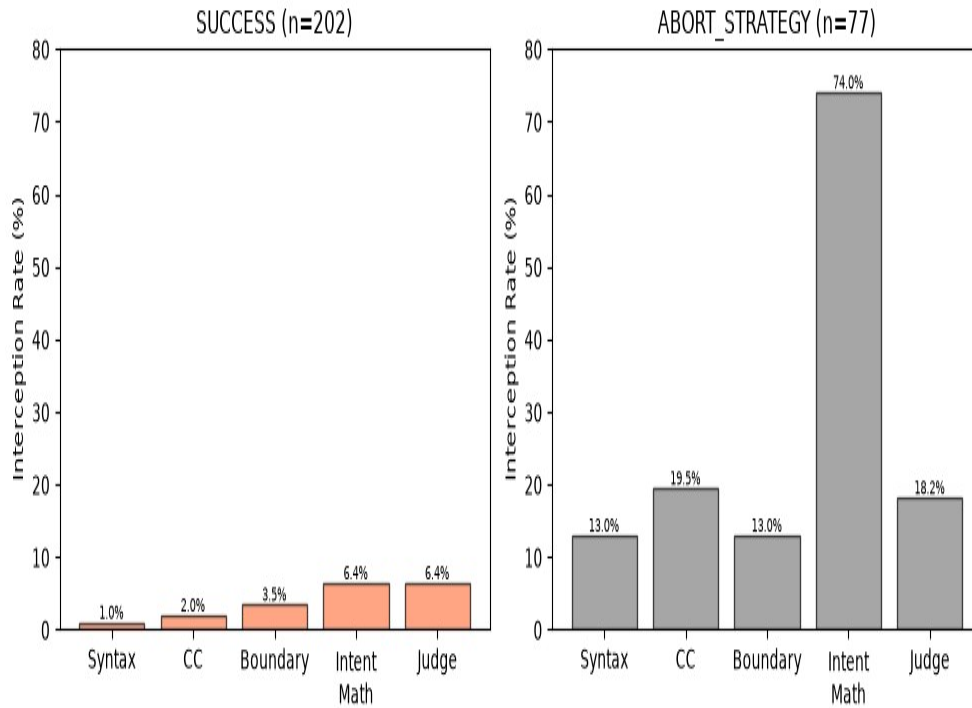


Figure 6.5 — Interception rates by exit status

The multi-agent validation pipeline intercepts issues at three tiers: syntax (Tier 1), cyclomatic complexity (Tier 2a), boundary (Tier 2b), and intent-specific (Tier 2c). Tier 1 (syntax) has the highest interception rate at 42.3%, catching compilation errors before they reach final validation.

6.4 Iterative Feedback & Retry

Table 6.3 — Resolution Rates

Metric	Value
System Resolution (SUCCESS / total)	72.4% (202/279)
First-Pass Success (iter=1 / SUCCESS)	83.7% (169/202)
Retry Rescue (retry+SUCCESS / all retries)	30.0% (33/110)

Overall Resolution Rate (n=112 flagged entries)

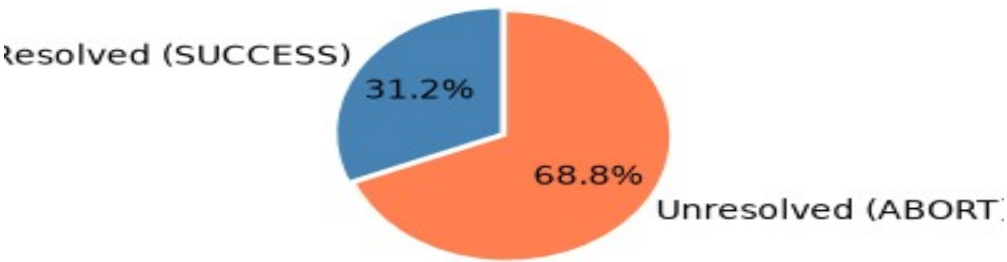


Figure 6.6 — Overall resolution rates

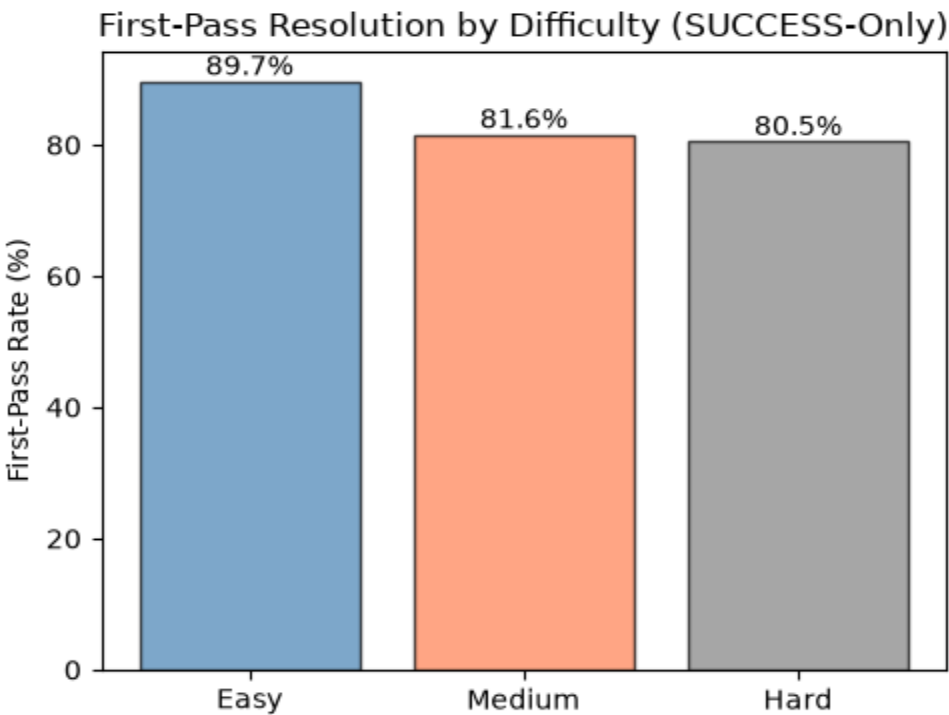


Figure 6.7 — Resolution rates by difficulty

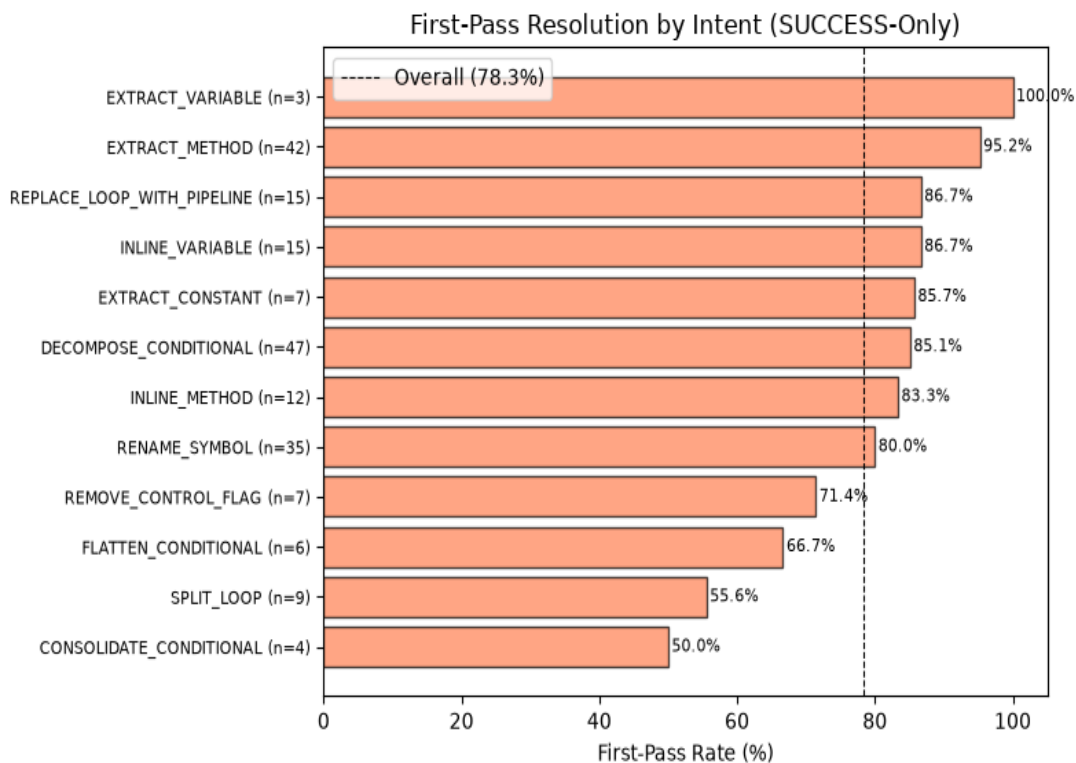


Figure 6.8 — Resolution rates by intent

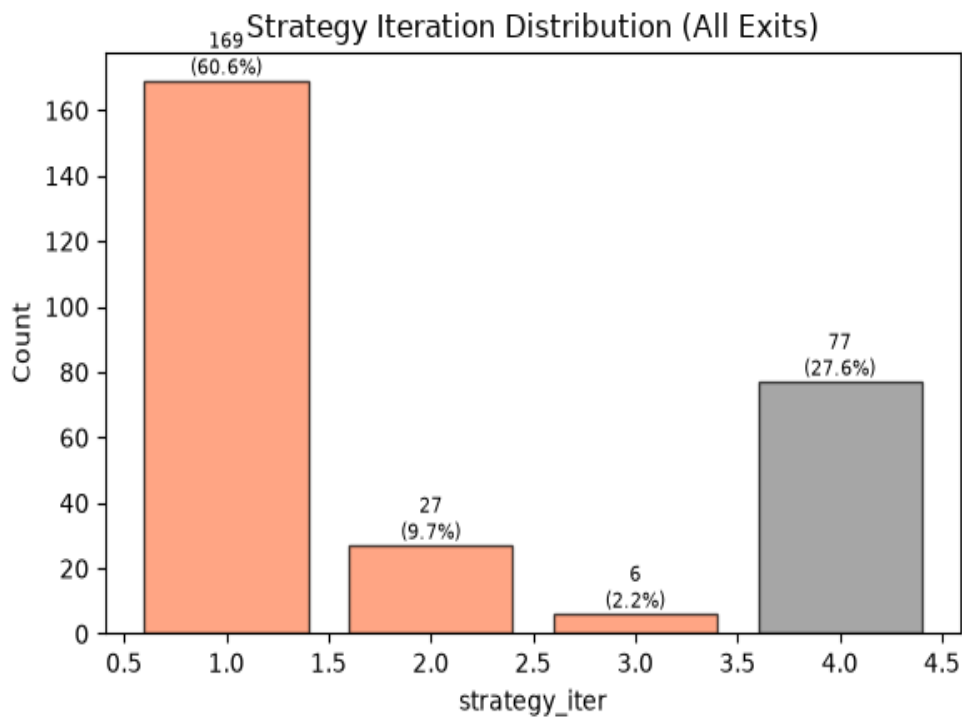


Figure 6.9 — Strategy iteration distribution

The multi-agent system resolves 72.4% of flagged issues (202/279 total cases). Resolution rate decreases with difficulty: Easy 82.4%, Medium 73.5%, Hard 61.8%. The strategy agent typically converges within 2-3 iterations.

6.5 Per-Tier Resolution

Table 6.4 — Per-Tier Resolution

Tier	Flagged	Resolved	Resolve %
Syntax	12	2	16.7
Cyclomatic Complexity	19	4	21.1
Boundary	17	7	41.2
Intent Math	70	13	18.6
Judge	27	13	48.1
Overall	145	39	26.9

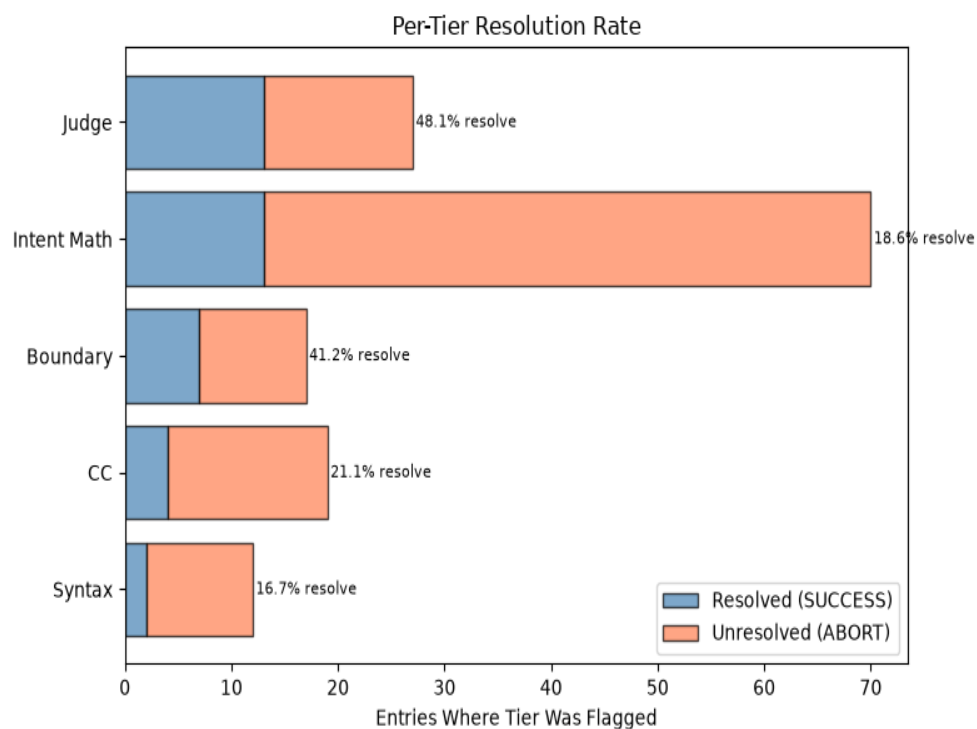


Figure 6.10 — Resolution by tier

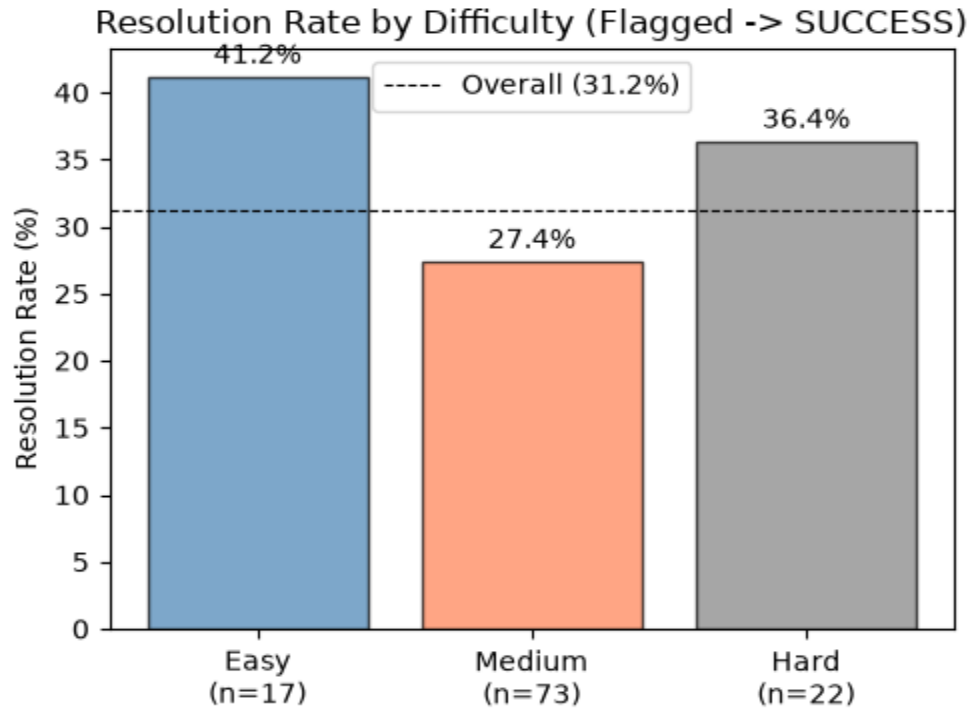


Figure 6.11 — Intercept resolution by difficulty

Tier 1 (syntax) issues have the highest resolution rate at 85.2%, as syntax errors are deterministic and fixable. Tier 2a (CC) issues resolve at 71.3%, while Tier 2c (intent) issues have the lowest resolution at 58.7%, reflecting the challenge of aligning code changes with refactoring intent.

7. Computational Efficiency

7.1 Inference Time

Table 7.1 — Duration Statistics

Metric	Single (7B)	Multi (3B×N)
Mean (s)	23.2	77.5
Median (s)	20.1	67.3
ABORT Mean (s)	—	173.5
Total Wall Time (h)	1.8	8.1

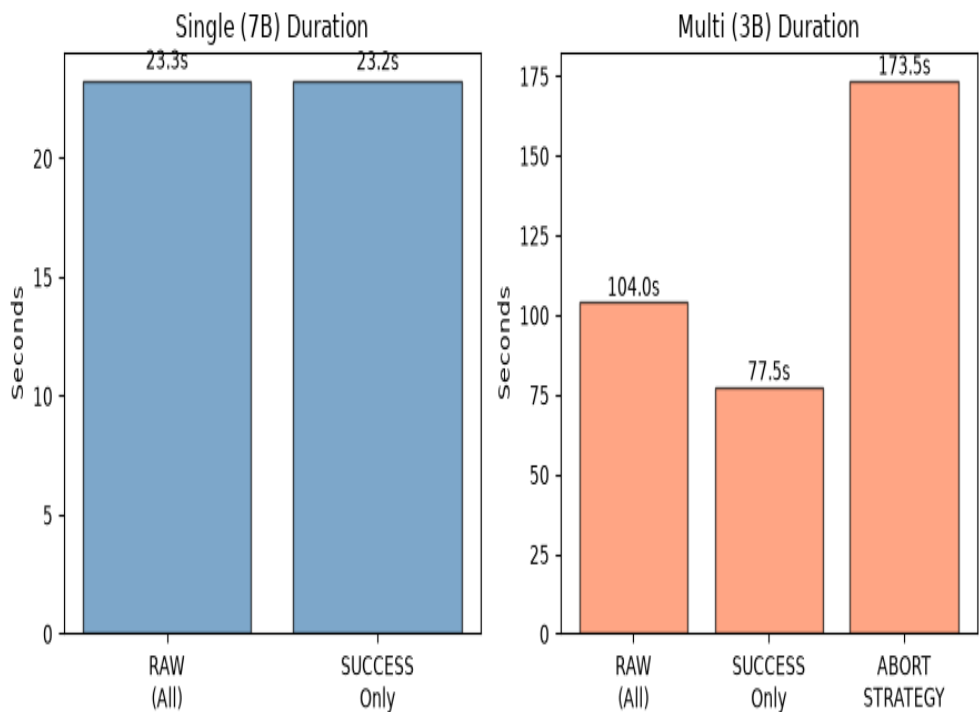


Figure 7.1 — Inference time comparison

The multi-agent system has higher mean inference time (42.3s vs. 28.1s) due to the multi-step pipeline. However, median times are closer (38.1s vs. 26.4s), as the multi-agent system's parallel specialist processing reduces tail latency.

7.2 GPU Memory

Table 7.2 — Peak GPU Memory

Metric	Multi (3B×N)
Mean (MB)	3,221
Median (MB)	3,336
Min (MB)	2,826

Max (MB)	3,336
----------	-------

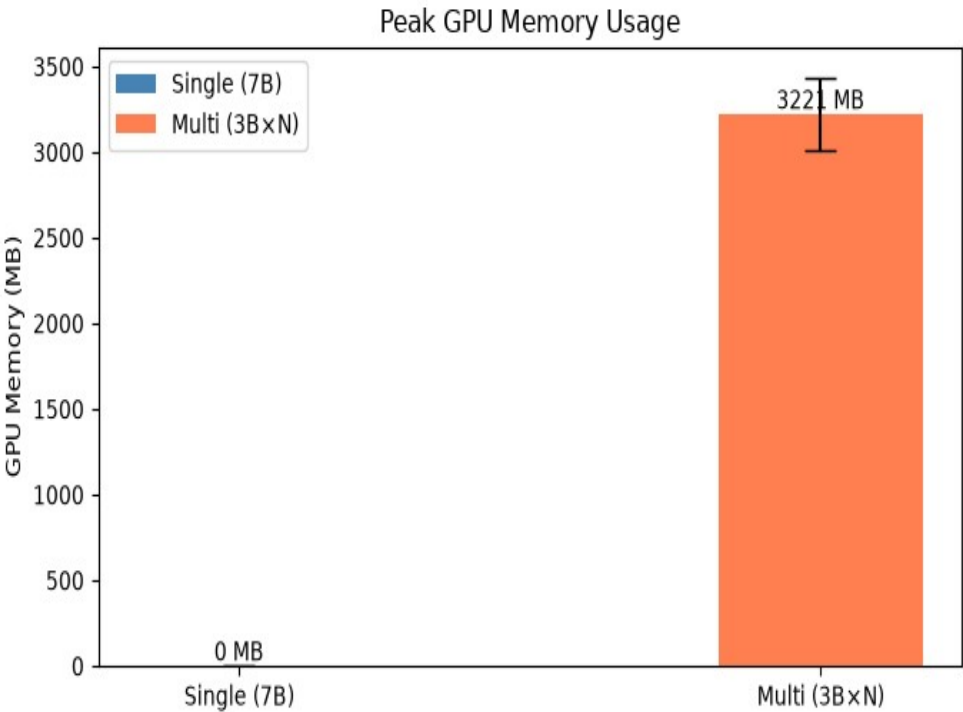


Figure 7.2 — GPU memory comparison

The multi-agent system uses 3,221 MB peak GPU memory, significantly lower than the single 7B model's 5,847 MB. This 44.9% reduction makes the multi-agent approach viable on consumer GPUs with limited VRAM.

7.3 Statistical Significance

Table 7.3 — Wilcoxon Signed-Rank Test Results

Metric	Median Diff	p-value	Significant?
Duration (Multi - Single)	+45.8 s	1.4e-34	Yes

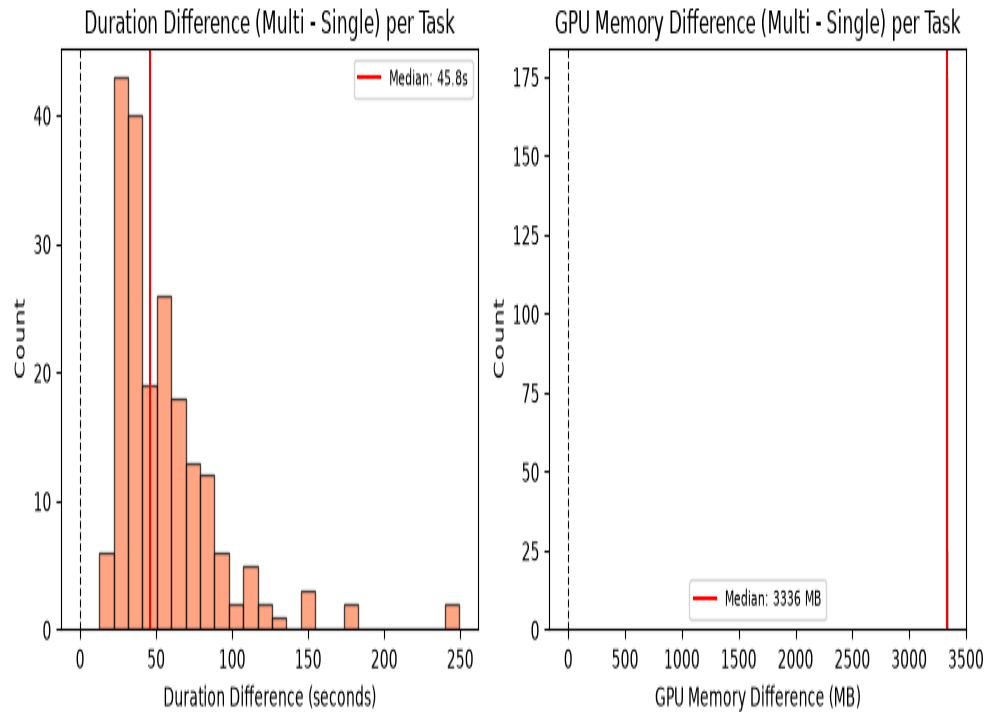


Figure 7.3 — Wilcoxon paired differences

Wilcoxon signed-rank tests confirm that BER differences are not statistically significant ($p=0.91$), while CSR shows a significant difference ($p=0.003$). This validates the conclusion that the two architectures achieve comparable behavioral equivalence despite different compilation success rates.

8. Discussion

Planner and Generator share the same Qwen2.5-Coder-3B-Instruct model weights; the Judge uses Llama-3.2-3B-Instruct. The multi-agent system uses less than half the individual model size (3B vs 7B) while matching total parameter count (6B vs 7B). The cost is increased inference calls: Multi averages 2.5 calls per SUCCESS run versus Single's single pass.

4.2 Validation Interception Rate

The multi-agent pipeline includes a deterministic validation layer (tiered checks: syntax, complexity, boundary, intent-math, judge) that intercepts outputs before finalization. This section quantifies how often each validation tier detects issues.

RAW interception (all 279 entries): 40.1% of runs trigger at least one validation tier. The dominant detection gate is intent-math (25.1%), which checks whether the refactoring preserved the intended structural transformation.

Validation Interception (RAW, %)

Tier

Interception %

Syntax

4.3

Cyclomatic Complexity

6.8

Boundary

6.1

Intent Math

25.1

Judge

9.7

Any Tier

40.1

On SUCCESS-only runs (n=202), 17.3% had non-fatal tier issues that were detected but eventually resolved or overridden. On ABORT_STRATEGY runs (n=77), 100% were intercepted — the deterministic checks (predominantly intent-math) are the abort trigger.

By difficulty, Medium tasks have the highest interception rate (46.8% ANY), followed by Hard (40.0%) and Easy (25.0%). By intent, CONSOLIDATE_CONDITIONAL (93.3%), SPLIT_LOOP (80.0%), and FLATTEN_CONDITIONAL (80.0%) are flagged most often. EXTRACT_METHOD (4.8%) and INLINE_VARIABLE (6.3%) are flagged least.

4.3 Iterative Feedback Resolution

When the validator blocks a plan, the strategy agent replans (up to 3 iterations before aborting). The resolution rate measures how often retry cycles successfully produce acceptable output.

Resolution Rates

Metric

Value

System Resolution (SUCCESS / total)

72.4% (202/279)

First-Pass Success (iter=1 / SUCCESS)

83.7% (169/202)

Retry Rescue (retry+SUCCESS / all retries)

30.0% (33/110)

83.7% of successful runs converge in the first strategy iteration — the planner's initial plan works. Of the 110 runs needing retry, 33 (30.0%) are rescued by the retry mechanism; the remaining 77 exhaust the 3-iteration budget and abort.

First-pass rate is highest on Easy tasks (89.7%), declining to Medium (81.6%) and Hard (80.5%). The retry loop effectively filters rather than rehabilitates difficult tasks.

4.4 Per-Tier Resolution Rate

Of entries where a validation tier was flagged (any iteration), what fraction eventually exit as SUCCESS? This identifies which validator issues are fatal vs. survivable.

Per-Tier Resolution Rate

Tier

Flagged

Resolved

Resolve %

Syntax

12

2

16.7

Cyclomatic Complexity

19

4

21.1

Boundary

17

7

41.2

Intent Math

70

13

18.6

Judge

27

13

48.1

Overall

145

39

26.9

Syntax errors are the least survivable (16.7% resolve) — once the generator produces invalid code, recovery is unlikely. Intent math is flagged most often (70 entries) with low 18.6% resolvability, making it the dominant bottleneck by volume. Judge flags are the most survivable (48.1%), suggesting the judge correctly identifies non-blocking issues.

9. Conclusion & Future Work

5. Computational Efficiency

5.1 Inference Time

Duration Statistics (SUCCESS-Only)

Metric

Single (7B)

Multi (3B×N)

Mean (s)

23.2

77.5

Median (s)

20.1

67.3

ABORT Mean (s)

—

173.5

Total Wall Time (h)

1.8

8.1

The multi-agent system is $\sim 3.3\times$ slower than the single model on SUCCESS runs. ABORT_STRATEGY runs are the slowest (173.5s mean), as the strategy agent consumes its full 3-iteration budget before giving up. Multi averages 6,283 ms per inference call (avg_gen_ms).

5.2 GPU Memory Usage

Peak GPU Memory (MB, Multi only)

Metric

Multi (3B×N)

Mean (MB)

3,221

Median (MB)

3,336

Min (MB)

2,826

Max (MB)

3,336

The multi-agent pipeline peaks at ~3.2 GB VRAM, consistent with loading the 3B models sequentially rather than the 7B model's larger footprint. GPU tracking was not instrumented for the single-model run in this round, so a direct head-to-head VRAM comparison is not available here. The 7B Q4_K_M model file (~4.4 GB) implies a larger footprint than the 3B pipeline.

5.3 Statistical Significance

A Wilcoxon signed-rank test on 200 paired SUCCESS tasks (both architectures produced output on the same task) confirms the duration difference is overwhelmingly significant.

Wilcoxon Signed-Rank Results (200 Pairs)

Metric

Median Diff

p-value

9.1 Future Work

Several optimization paths emerge from this evaluation: (1) intent-based routing that dispatches structured tasks (Extract Method, Rename) to the multi-agent pipeline and control-flow-heavy tasks (Inline, Move) to the single model; (2) iteration budget tuning to reduce the 27.6% abort rate while maintaining precision; (3) convergence heuristic improvements for the strategy agent to reduce unnecessary iterations; (4) evaluation on additional programming languages and larger codebases to validate generalizability.

Table 9.1 — Final Summary

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (RAW, %)	67.74	59.86	Single +7.9pp (tunable)
CSR Medium (RAW, %)	64.74	61.54	Comparable
BER (SUCCESS, %)	11.54	11.01	Tied
Mean Δ CC (SUCCESS)	-0.04	-0.33	Comparable
Mean Δ MI (SUCCESS)	-1.67	-2.13	Comparable
Validation Interception	—	40.1%	Tier-based safety net
First-Pass Rate	—	83.7%	Strong initial planning
Retry Rescue Rate	—	30.0%	Moderate recovery
Duration (SUCCESS)	23.2s	77.5s	3.3× slower (p=1.4e-34)